

Средства симуляции ЦП и ОС

Лекция №9

Державин Андрей
Шурыгин Антон

Scope of simulation

Поиграем?

- Написали интерпретатор (и бинарный транслятор)
- Что он умеет?
 - Выполнять арифметические операции над регистрами
 - Совершать условные/безусловные переходы
 - Работать с памятью
 - **Имитировать** взаимодействие с ОС (как?)
- *Симулятор Режимы Приложения (App Mode Simulator)*
- *Симулятор Режимы Пользователя (User Mode Simulator)*



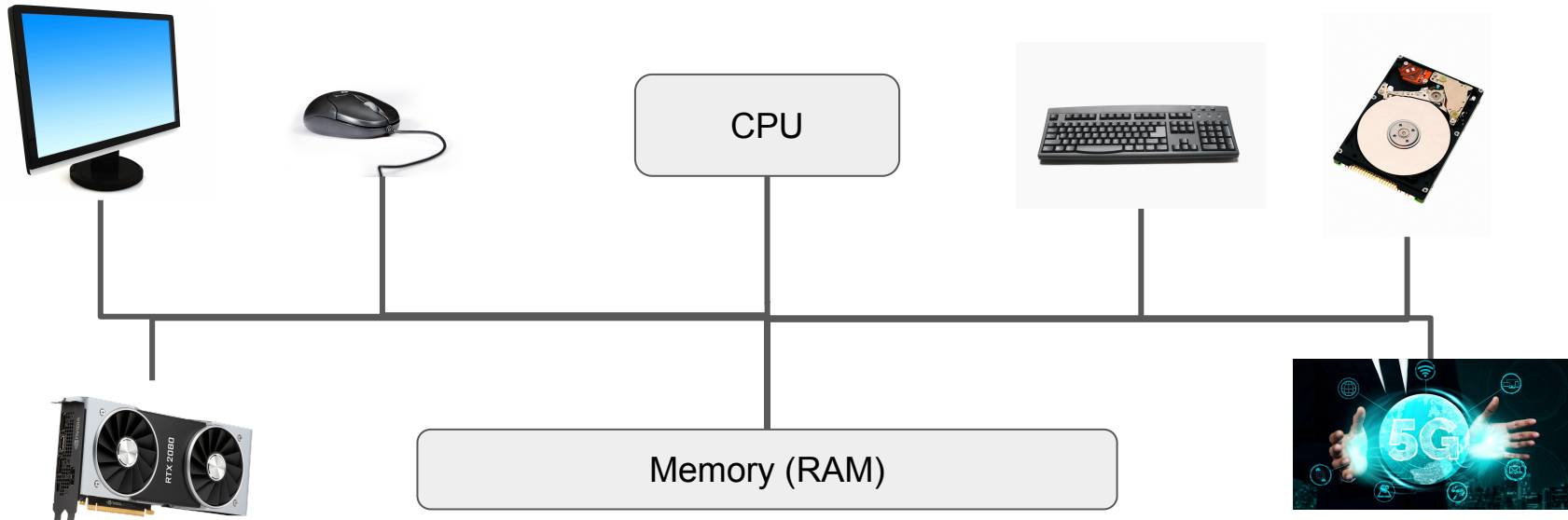
Extend Scope

- Умеем симулировать пользовательские приложения
- Достаточно для производства нового чипа?
 - Кажется, что нет
- Пользовательские программы работают внутри ОС
 - ОС - (почти) такая же программа
- Программа взаимодействует с внешними устройствами через ОС
- Хотим исполнять код операционной системы
- Что для этого нужно?

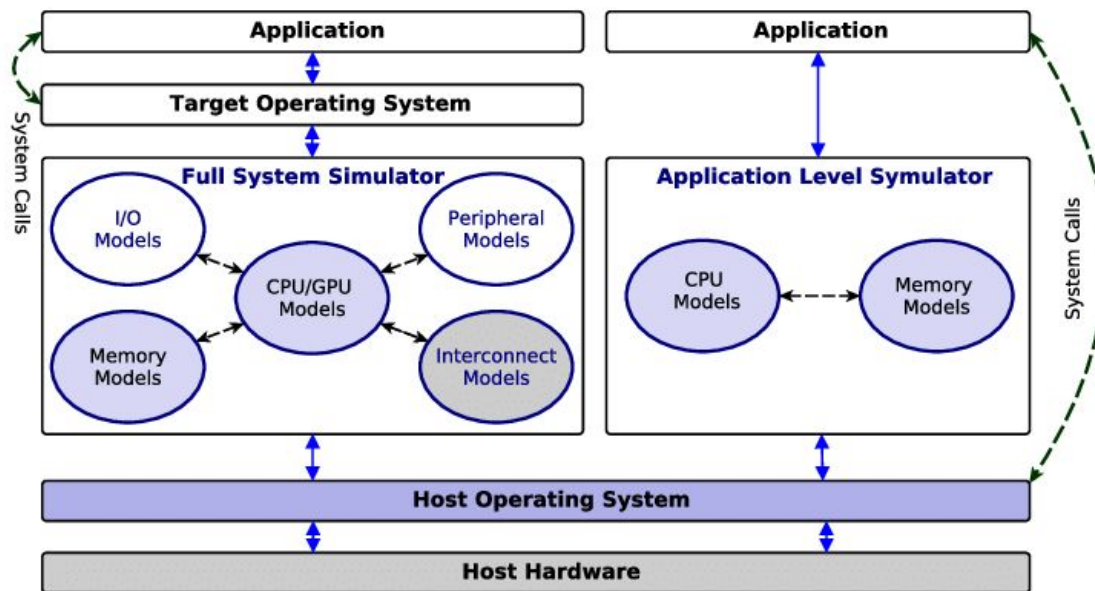


Extend Scope

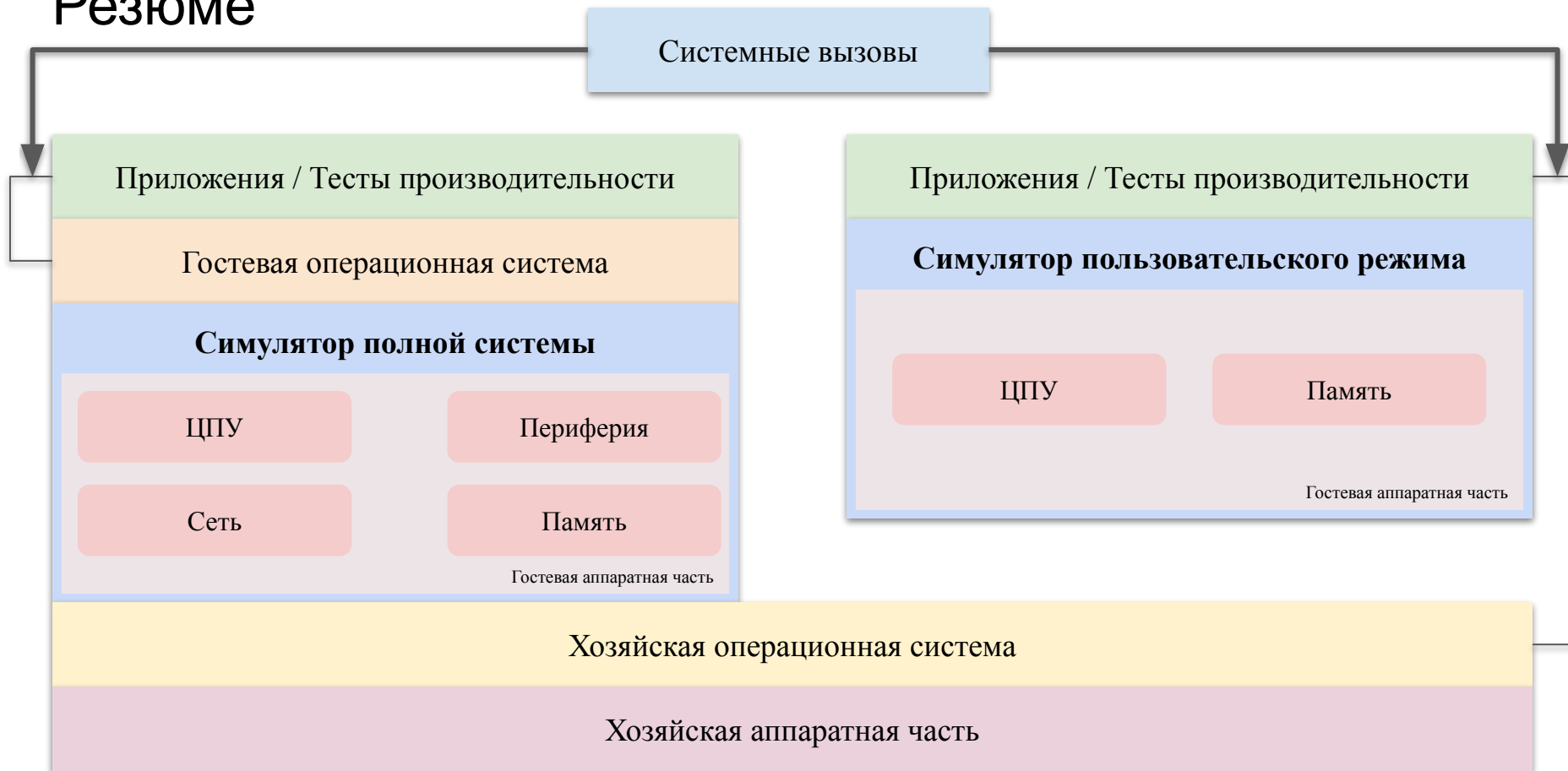
- Современные вычислительные системы это не только CPU
 - И не только один CPU
 - А еще *PU
 - Или даже не PU...
- Симуляция Полной Платформы (Full System Simulation)



Резюме



Резюме

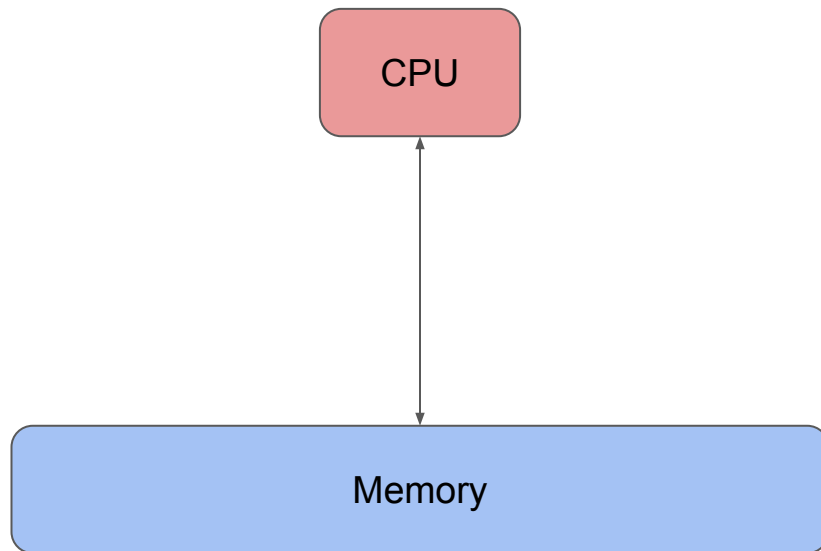


А как моделировать что-то кроме PU?

Симуляция полной платформы

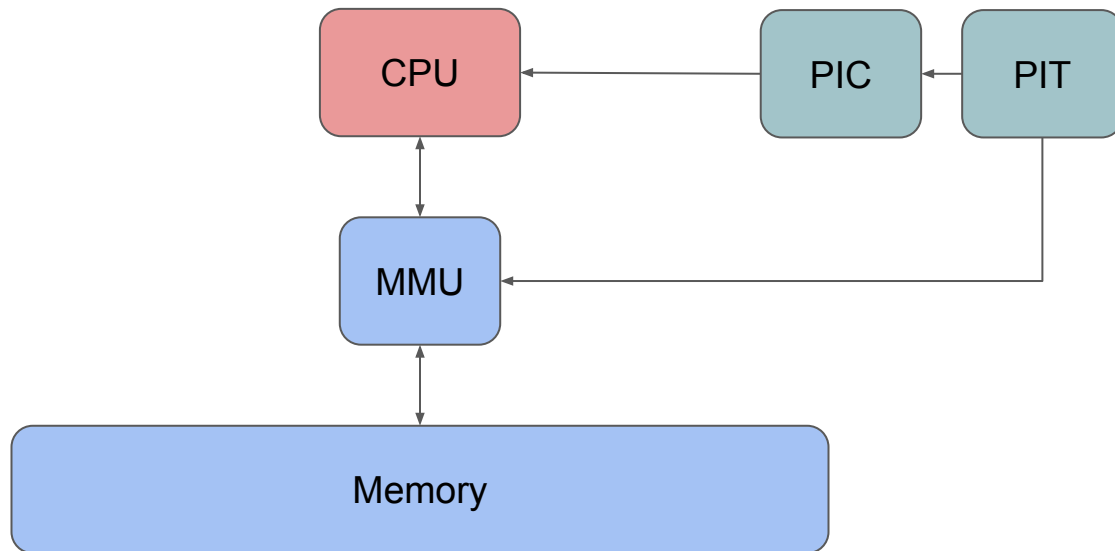
Симуляция полной платформы

- Какие компоненты есть в нашей модели?



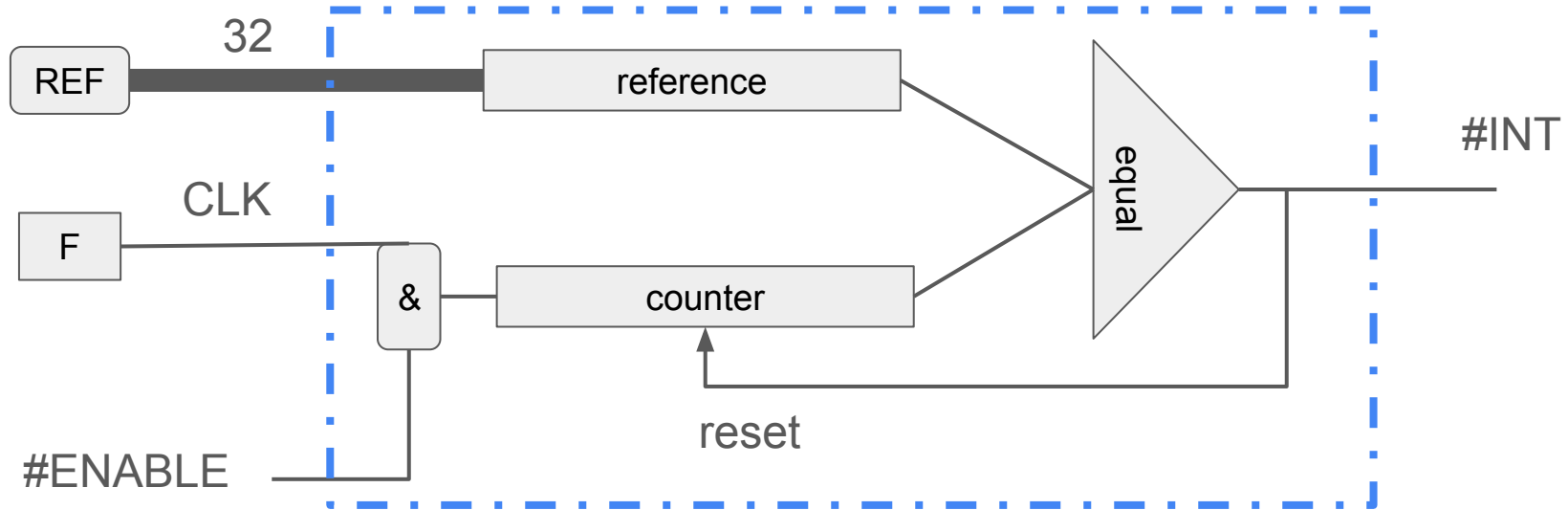
Симуляция полной платформы

- PIC - контроллер прерываний, PIT - таймер



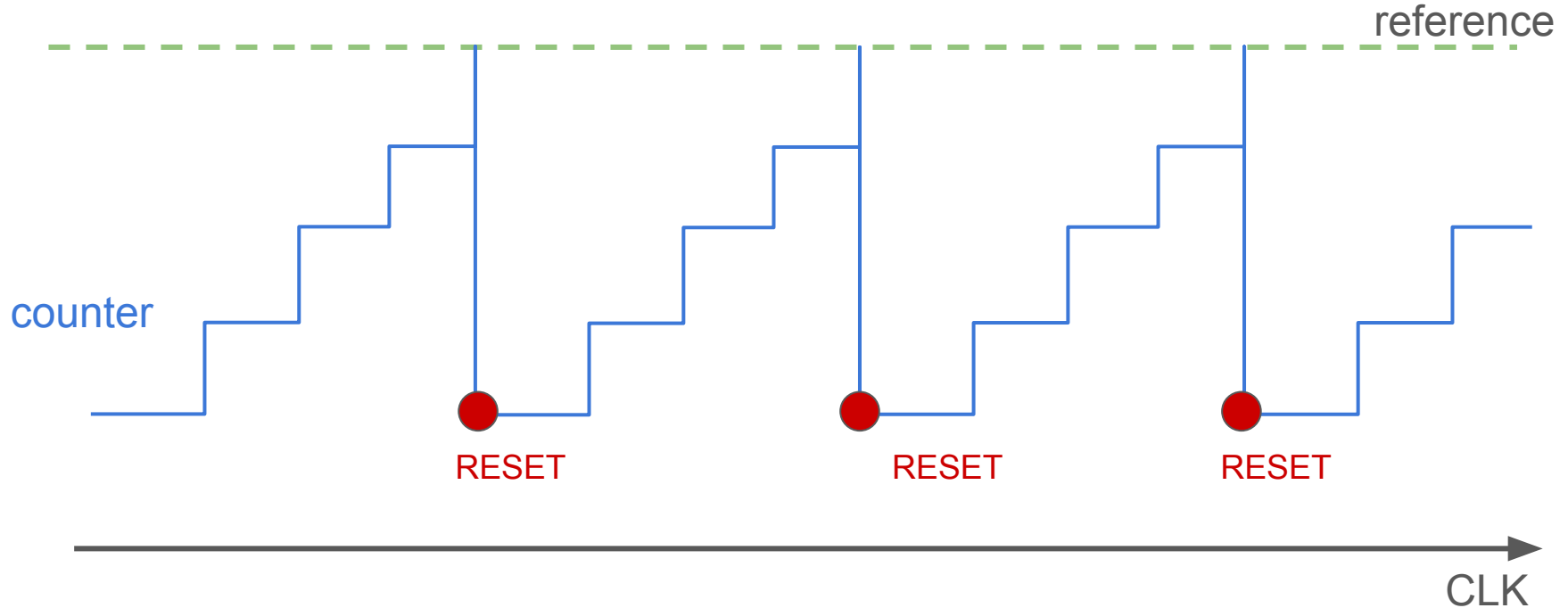
Симуляция полной платформы. Таймер

- Часто встречающееся устройство
- Задача - сгенерировать сигнал (прерывание) по истечении времени



Симуляция полной платформы. Таймер

- Как работает таймер?



Симуляция с фиксированным шагом

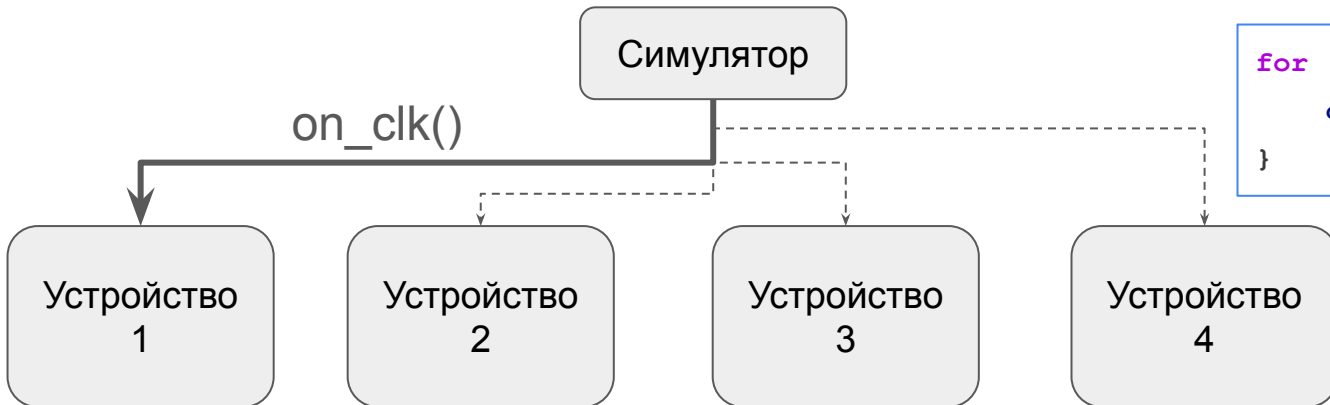
- Как написать модель таймера?
- Пойдём по-честному

```
struct Timer : IDevice {  
    bool enable{true};  
    unsigned counter;  
    unsigned reference;  
    void on_ref(unsigned new_ref) { reference = new_ref; }  
    void on_reset() { counter = 0; }  
    void on_clk() override {  
        if (enable)  
            counter += 1;  
        if (counter == reference) {  
            raise_int();  
            on_reset();  
        } else  
            lower_int();  
    }  
};
```

Симуляция с фиксированным шагом

- На каждом такте обновляем внутренне состояние устройств
- Всё хорошо?
 - Устройств может быть **очень** много
 - $F \sim 10^7$ Гц
 - `reference > 1000`
 - `ENABLE` меняется с частотой < 0.01 Гц

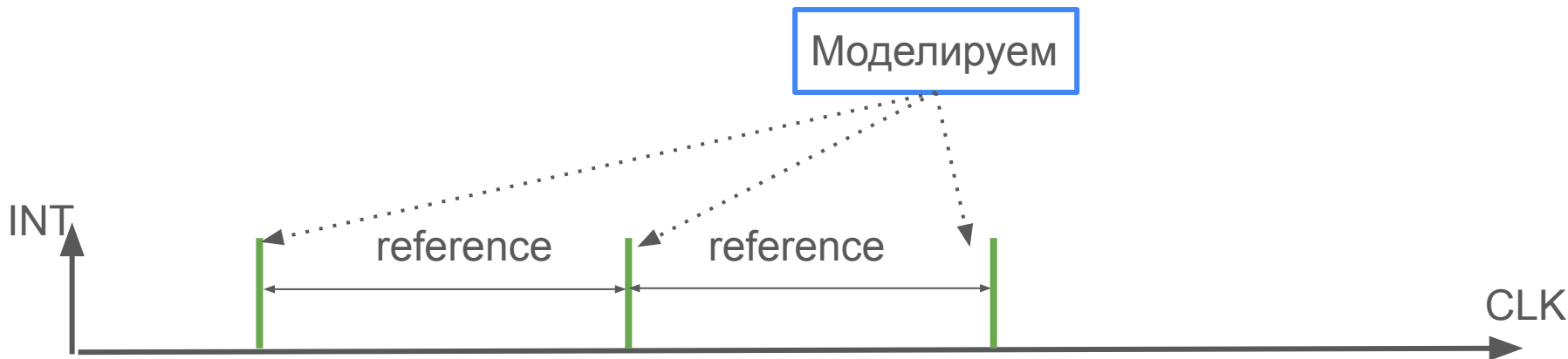
$f_{INT} \sim 10^4$ Гц
 $F_{cpu} \sim 10^9$ Гц
 $t_{INT} \sim 10^5$ тактов



```
for (auto *device : devices) {  
    device->on_clk();  
}
```

Симуляция с фиксированным шагом

- `on_clk()` обладает видимым эффектом каждые 100'000 вызовов
- Остальные 99'999 вызовов меняют только внутреннее состояние
- Что нам интересно в контексте симуляции ОС?
 - Снова принцип абстракции
- Как выглядит график внешних эффектов?



Симуляция полной платформы. Таймер

- Достаточно вовремя выставить #INT
- Храним события в списке
 - *Очередь событий*

```
struct Event {  
    std::uint64_t time_delta;  
    IDevice *dev;  
    void (*function) (IDevice *);  
};
```

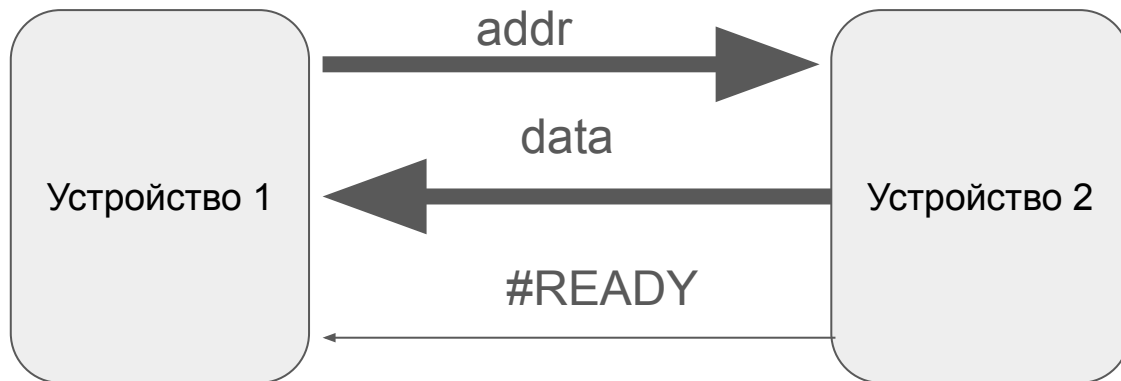
```
for (auto &event : events) {  
    event.function(event.dev);  
    tsim += event.time_delta;  
}
```

Симуляция полной платформы. Таймер

- Что поменялось в модели?
- Поступление сигнала CLK не моделируется
 - Не тратим время на обработку внутренних изменений
- Что происходит при $ENABLE = 0$?
 - Все будущие события удаляются
- Что происходит при изменении reference?
 - Удалить будущие события
 - Добавить новые, с обновлённым периодом
- Такая схема легко расширяется на множество устройств

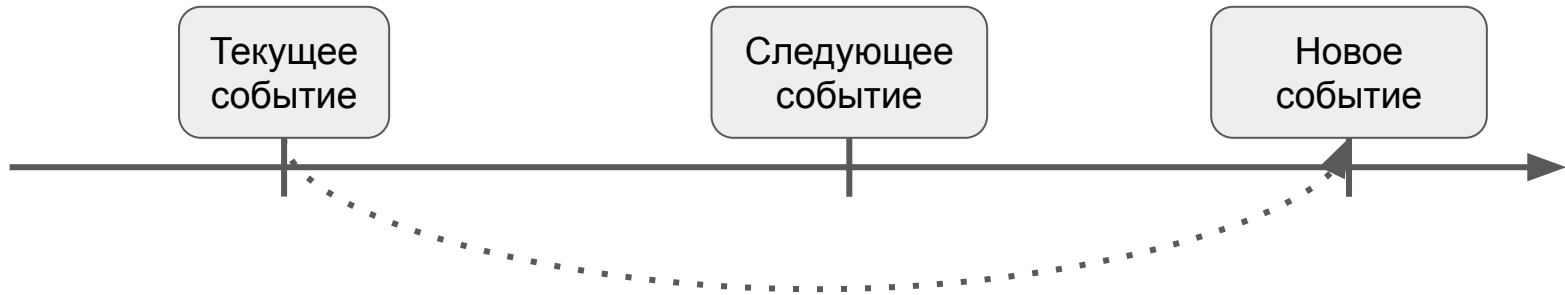
Отложенный ответ

- Попробуем фиксированный шаг
 - На **каждом** шаге проверяем READY...
- Используем *очередь событий*
 - Вычисляем время ожидания dt
 - Помещаем событие READY на время $t + dt$
 - Продолжаем симуляцию
- Чем то похоже на таймер?



Симуляция полной платформы. Event-driven

- Обобщим модель таймера с очередью событий
- Симуляция дискретных событий (Discrete Event Simulation, DES)
- Событие - изменение состояний устройств
- Симулируемое время двигается по событиям (скачками)



Симуляция полной платформы. Event-driven

- Реализуем очередь событий
- Какую структуру данных возьмем?
 - Очередь с приоритетами
 - Приоритет - время события
- Принципы такой очереди
 - Добавление
 - Добавляем событие в соответствии с приоритетом (время)
 - При равенстве - лучше придерживаться детерминизма
 - События независимы
 - Нельзя добавлять в прошлое
 - Удаление
 - Отмена события

Симуляция полной платформы. Event-driven

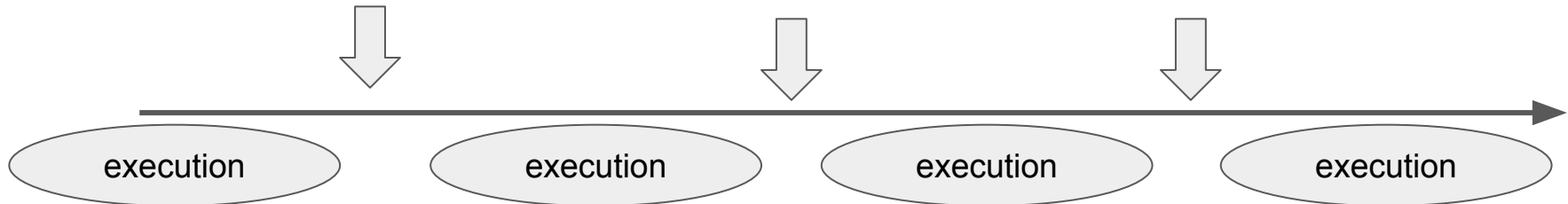
- Что общего у устройств подобного рода?
 - Управляются внешними событиями
 - Меняют однократно свое состояние на них
- Давайте моделировать CPU таким же образом!
 - Будет медленно
- Кажется этот метод не подходит...

Симуляция полной платформы. Execution-driven

- На самом деле, вы уже писали такие модели
 - Интерпретатор, ДДТ...
- Модель совершает действие каждый такт, пока не будет прервана внешним событием

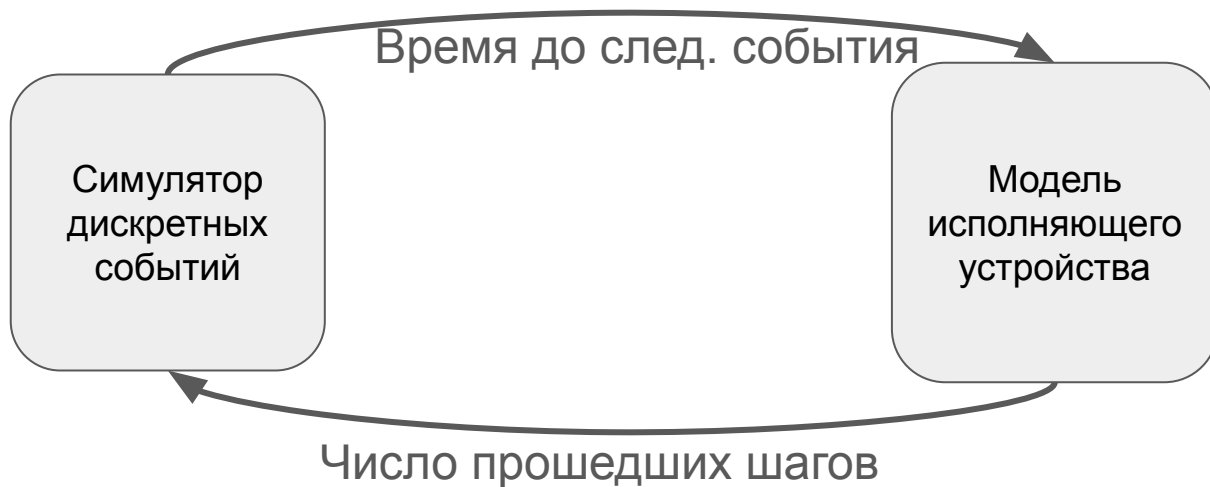
Симуляция полной платформы. Два класса моделей

- Event-Driven
 - Состояние меняется редко и асинхронно
 - Взаимодействие в виде “запрос-отклик”
 - Можно представить в виде “черного ящика”
- Execution-Driven
 - Состояние меняется каждый такт (почти)
 - Внешние события случаются редко
 - Внутренние процессы устройства представляют интерес



Симуляция полной платформы. Два класса моделей

- Платформа состоит из исполняющих и неисполняющих устройств
- Симуляция разных типов устройств перемежается
- Управление передаётся по условиям



Ссылки

- [Simulation Book, Grigoriy Rechistov, 2023](#)
- Zarrin, Javad & Aguiar, Rui & Barraca, João. (2017). Manycore simulation for peta-scale system design: Motivation, tools, challenges and prospects. Simulation Modelling Practice and Theory. 72. 168-201. 10.1016/j.simpat.2016.12.014.